Algorithms for massive data

Market-Basket Analysis Project

Alessandro Macchi
MSc in Data Science for Economics
Università degli Studi di Milano

March 10, 2026

Contents

1 Introduction **1**

2 Data Exploration and Preprocessing **1**

3 Algorithm Implementation **1**

4 Scalability Proof **3**

5 Experiments **4**

6 Discussion **5**

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

1 Introduction

The primary objective of this project is to perform a Market-Basket Analysis which aims to identify frequent itemsets within a given dataset. Specifically, the focus will be on discovering the most frequent pairs of actors who worked on the same productions.

Three different algorithms have been implemented and will be presented in the following sections: a-priori, PCY and SON.

2 Data Exploration and Preprocessing

The object of this analysis is the IMDB Movies Dataset[1], which collects the top 1000 movies and tv series on the well-known IMDB website.

In addition to the title, genres, runtime and rating, the dataset includes the four most relevant actors for each film, under the columns $Star1$, $Star2$, $Star3$ and $Star4$, which will be the focus of our analysis.

The dataset, originally stored as a CSV file, was loaded as a Pandas DataFrame. The unnecessary columns were dropped and a subset with only the actor's variables, called $stars_df$, was created.

After noticing that all these columns do not have null values, the data were organized into a list named *baskets*; each entry in this list is a sub-list that groups the associated actors for a specific movie.

For the first experiments, a sample of the original subset (30%) was used, to test the algorithms without waiting for the computation of the whole data. Even though in this case we are not dealing with a massive dataset, this is a very common best practice for this kind of datasets.

3 Algorithm Implementation

The simplest algorithm for Market-Basket Analysis is the a-priori algorithm, which identifies all the most frequent itemsets in just two passes over the entire dataset. The implementation can be divided into three steps:

[1] https://www.kaggle.com/datasets/harshitshankhdhar/imdb-dataset-of-top-1000-movies-and-tv-shows

1. First Pass: The data are scanned for the first time and two tables are created, one to link the item name with a unique "item id", for memory efficiency, and the other to count the occurrences of every single item over the entire dataset.

2. Between Two Passes: Frequent items, which are the single items that have a number of appearances at least equal to the support threshold s, are identified and assigned a new numbering to distinguish them from the infrequent items. This filtering is possible because the baskets are monotone, so an infrequent item cannot be part of a frequent itemset.

3. Second Pass: All pairs are counted, but just considering the items that have been found to be frequent. The ones with a number of occurrences at least equal to the support threshold s are frequent pairs. To avoid situations in which pairs (1, 2) and (2, 1) have different counters due to order, all the items have been sorted according to their item id.

However, the a-priori algorithm can suffer from memory problems when applied to massive datasets, that is why there exist some variants. The algorithm of Park, Chen and Yu (PCY) exploits the observation that there may be much unused space on the main memory on the first pass. The implementation can be divided into three steps:

1. First Pass: in addition to the operations executed during the first pass of the a-priori, a hash function is implemented to map every possible pair to a specific bucket in a hash table. The pairs are not stored in that bucket, but each one of them maintains a counter that is incremented every time a pair hashes to it.

2. Between Two Passes: After frequent items have been identified, a smaller bitmap substitutes the bucket table to distinguish between frequent and infrequent buckets. In fact, if a pair is in a frequent bucket (i.e., bitmap[bucket] = 1), we can tell nothing about its frequency, but pairs inside infrequent buckets are surely infrequent and can be ignored during counting.

3. Second Pass: Exactly like in the a-priori, this pass over data is used to count over the items that satisfy both the condition established during the previous step and identify the most frequent pairs.

Finally, the last algorithm implemented was developed by Savasere, Omiecinski and Navathe (SON). The real upside of this instance is the possibility of computing

through parallelization, which tremendously accelerates computations when more than one machine is available. In order to perform this operation efficiently, Spark was used. This design was developed to run MapReduce programming and horizontally scale the execution over massive datasets. Even though the data are not massive, this toy example shows and implements the same techniques that could scale in case of larger datasets. The code can be divided into four steps:

1. Chunks Creation: The dataset is divided into smaller chunks, such that each is a fraction p of the original dataset. Each chunk is then transformed into a Resilient Distributed Dataset (RDD), the fundamental structure behind Spark.

2. A-priori: The a-priori algorithm is executed on each chunk, applying a support threshold of ps. The outputs are the candidate pairs for that chunk. In this step, total occurrences are not counted.

3. First MapReduce: All the candidates from the a-priori output are transformed in a set of key-value pairs (C, 1) and grouped by key, without counting the occurrences. The final output is a set of unique pairs of candidates.

4. Second MapReduce: Passing through the entire dataset, a list of all candidate pairs obtained from the first MapReduce is generated, and their occurrences are counted. Finally, we consider only those that have support at least s.

4 Scalability Proof

In order to assess scalability, every algorithm has been tested on 30%, 60% and 100% of the data. The execution time[2] has been compared between all the algorithms over a series of runs. The results are presented in the following table.

Fraction of Data	**A-priori Time (s)**	**PCY Time (s)**	**SON Time (s)**
30%	0.001700	0.008245	0.808833
60%	0.007515	0.015161	0.810979
100%	0.018267	0.018280	0.817121

Table 1: Execution times of A-priori, PCY, and SON algorithms across different data fractions.

[2]Note that the exact time depends on a series of factors and that it may slightly change

A-priori was the fastest across all data sizes, doubling the overall time each step. Given the small dimension of the dataset, it scaled highly effectively without running into bottlenecks.

PCY consistently took longer to execute, because of the extra step during the first pass in which it hashes all the single items. Due to the small size of the dataset, this attempt is not very effective, but it removes some candidate items, improving the time, if compared to the a-priori, when dealing with the entire dataset.

SON had the highest absolute execution times, but demonstrated exceptional scalability. As the data fraction increased, the execution time remained virtually constant. In contrast, a-priori and PCY saw their execution times increase significantly as data grew. This confirms that the SON bottleneck is the "fixed costs" of parallelization and the Spark framework.

Finally, all the algorithms have been tested and they consistently retrieve the same frequent pairs across all the different samples used.

5 Experiments

Firstly, the a-priori algorithm is implemented. For all algorithms, a support threshold equal to 3 has been chosen. This is because it was empirically noticed that it selects the 25 most frequent pairs of the possible 6000 pairs available (0.4%), while an additional unit of support would be too restrictive.

For the PCY algorithm, other than the support threshold, the other parameter is the number of buckets for the hashing section. Based on Example 6.9 of Rajaramanu & Ullman (2011)[1], a total number of 5000 buckets was chosen, in order to remove 50% of the candidate pairs, but avoiding an exaggerated quantity that would slow down the entire algorithm.

Finally, the SON algorithm has as input the number of chunks. Although the dataset could theoretically be partitioned into a larger number of chunks, doing so would make initial filtering ineffective. Specifically, selecting three or more chunks would result in a local threshold of $ps \leq 1$. Therefore, all the single items and pairs would pass the a-priori section, since being in the dataset is sufficient to fulfill these conditions. By restricting the division to two chunks, the local threshold becomes $ps = 3/2$, ensuring that only some single items and pairs pass this filtering part.

Except for the SON algorithm, which relies on Spark's operations, itemsets are tracked using the triple method, which stores the candidate pairs using a triple {item i, item j, count c}. This approach can be implemented, like in this case, with a dictionary, where the key is a tuple containing the two items and the value is the count. This method was chosen over others, like the triangular matrix, because data can be sparse and an actor that appears in just a few films does not have to be paired to all the other actors, wasting space.

6 Discussion

As mentioned above, the support threshold was chosen to filter the 25 most frequent pairs out of the possible 6000. This was done to make the result readable and to gain some insights from it.

The frequent itemsets identified can be divided into two distinct groups:

1. Franchise pairs: The vast majority of the pairs comes from films series, like Harry Potter (Daniel Radcliffe & Emma Watson, ...), Star Wars (Mark Hamill & Harrison Ford, ...) and the Marvel movies (Robert Downey Jr. & Chris Evans, ...).

2. Famous duos: Legendary actors who worked together like Al Pacino & Robert De Niro, Robert De Niro & Joe Pesci, ...

Regarding the algorithms and considering the previously discussed scalability, the advantages of PCY and SON are not realized because we are not dealing with a truly massive dataset. In this situation, a-priori wins just by being the simplest algorithm, but would have some problems dealing with millions of possible pairs instead of just a few thousand.

References

[1] Anand Rajaraman and Jeffrey David Ullman. *Mining of Massive Datasets*. Cambridge University Press, 2011. Available at `http://www.mmds.org/`.